

Lab 4

Process and Fork

Process

- **Processes** are primitive units for allocating system resources.
- Each process has its own **address space**.
- Processes **execute programs independently**, with their own copy of the program.
- Processes are organized **hierarchically**, with **parent** and **child** processes.
- Each process has a **unique process ID**.
- Processes are created using the **fork()** system call.

Monitoring Processes (ps command)

```
nour@WIN-2LMLHM6VJV6:~$ ps
  PID TTY          TIME CMD
 17790 pts/4        00:00:00 bash
 17895 pts/4        00:00:00 ps
```

```
nour@WIN-2LMLHM6VJV6:~$ ps -ly
 S    UID      PID  PPID  C PRI  NI   RSS   SZ  WCHAN  TTY          TIME CMD
 S   1000   17790  17789  0  80   0  5216  1551 do_wai pts/4        00:00:00 bash
 R   1000   18047  17790  0  80   0  1644  1869 -      pts/4        00:00:00 ps
```

Monitoring Processes (pstree command)

```
nour@WIN-2LMLHM6VJV6:~$ pstree
init--init--init--16*[cpptools-srv--15*[{cpptools-srv}]]
      |           |
      |           +--sh--sh--sh--node--node--3*[bash]
      |                   |               |
      |                   |               +--13*[{node}]
      |                   |               +--node--12*[{node}]
      |                   |               +--node--cpptools--17*[{cpptools}]
      |                   |                       |
      |                   |                       +--15*[{node}]
      |                   |               +--10*[{node}]
      |
      +--2*[init--init--node--6*[{node}]]
      +--init--init--bash--pstree
      +--2*[{init}]
```

Monitoring Processes (top command)

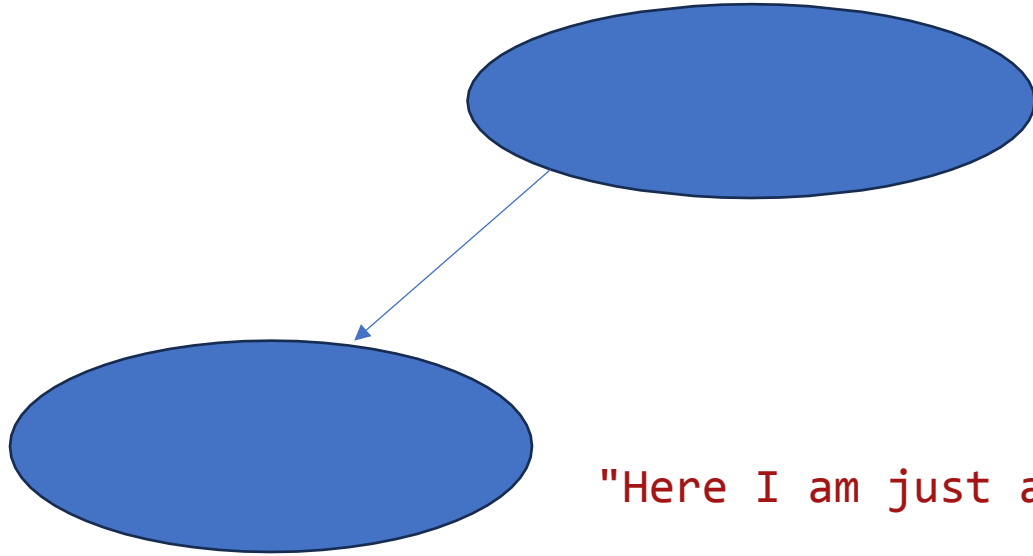
Simple Process Creation

- The fork function make a system call to create a new process.

Multiple Fork

- How many times each print statement will be executed?

Multiple Fork

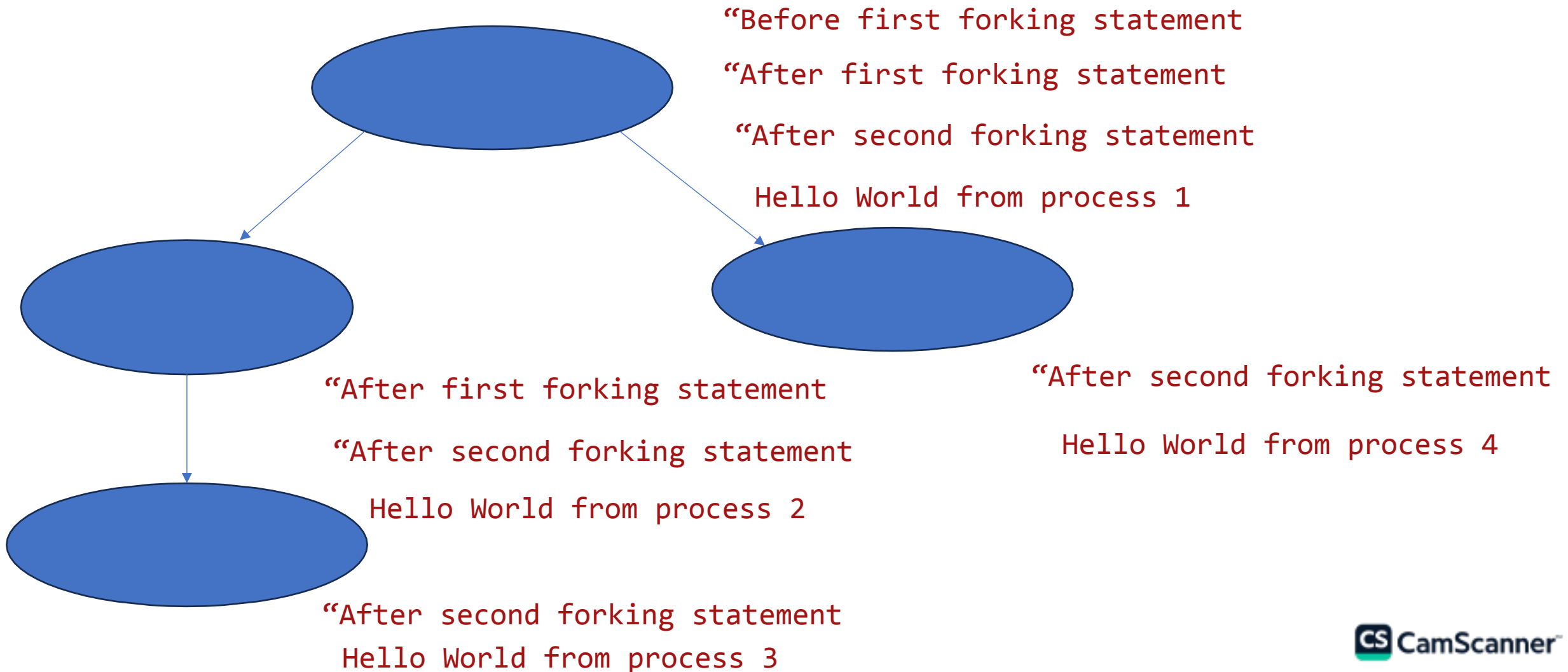


"Here I am just before the first forking statement

"Here I am just after the first forking statement

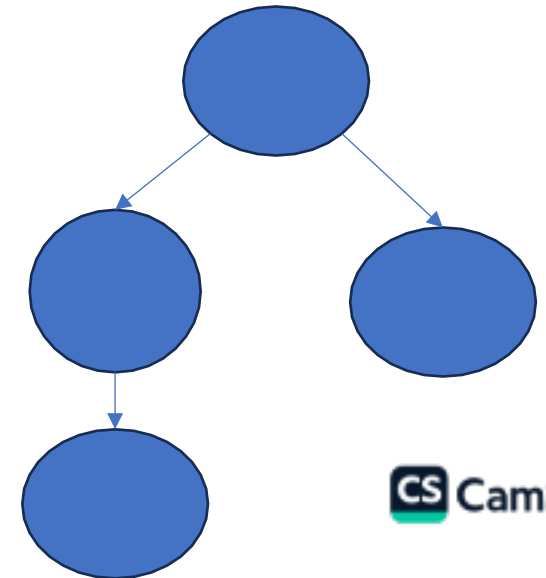
"Here I am just after the first forking statement

Multiple Fork



Multiple Fork

- Using pstree command to view the process tree



Fork Return Value

- The fork function creates a new process.
- If process creation **fails**, the **parent process** receives a return value of **-1**, and no child process is created.
- If **successful**, the **child process** receives a return value of **0**, while the **parent process** gets the **child's process ID**.
- There is **no guarantee** which process will execute first.

Control Execution Flow

Control Execution Flow

Parent Process

Child Process

pid : 3723

pid : 0



wait(), exit(), sleep()

wait() System Call:

- The wait() system call is used in Unix-like operating systems to manage child processes.
- When a parent process calls **wait()**, it blocks until one of its child processes exits or receives a signal.
- After a **child** process **terminates**, the **parent continues** its execution.
- From the include library “sys/wait.h”

exit() Function

exit() Function:

- The exit() function is used to terminate a process.
- It is available in both C and C++.
- When a process calls exit(), it immediately terminates.
- The function takes an optional integer argument (exit status) that indicates why the process.

sleep() Function

sleep() Function:

- The **sleep()** function causes a process to **pause execution** for a specified duration.
- It is useful for introducing delays or timing intervals.

Child Process Behavior

Orphan

- An **orphan process** is a process that continues to execute even after its parent process has terminated or exited.
- Orphan process are adopted by the **init** Process.

Orphan

Orphan



Zombie

- Zombie process is a process that has completed execution (via the exit system call) but still has an entry in the process table.
- Essentially, it is a process in the “terminated state”.
- Zombies exist because the parent process needs to read its child’s exit status before removing the child’s entry from the process table.

Zombie

Zombie

Task 1

Task2